

Figure 20. Left When training the helix with 80% of a values and activation patching with the other 20% of a values, we see that the helix and circular fits still outperform the PCA baseline. *Middle* We randomize a while fitting the helix and see that the randomized helix is not causally relevant. *Right* We show that our results for fitting the a token can be extended to fitting the b token on the prompt $a + b$ with $\text{helix}(b)$.

Table 3. GPT-J's task performance with corresponding domains, prompts, and accuracies.

Task	**Domain**	**Prompt**	**Accuracy**
$a - 23$	$a \in [23, 99]$	Output ONLY a number. a-23=	89.61%
$a//5$	$a \in [0, 99]$	Output ONLY a number. a//5=	97.78%
$a * 1.5$	$a \in [0, 98]$	Output ONLY a number. a*1.5=	80.00%
$a \mod 2$	$a \in [0, 99]$	Output ONLY a number. a modulo 2=	95.56%
$x - a = 0, x =$	$a \in [0, 99]$	Output ONLY a number. x-a=0, x=	95.56%

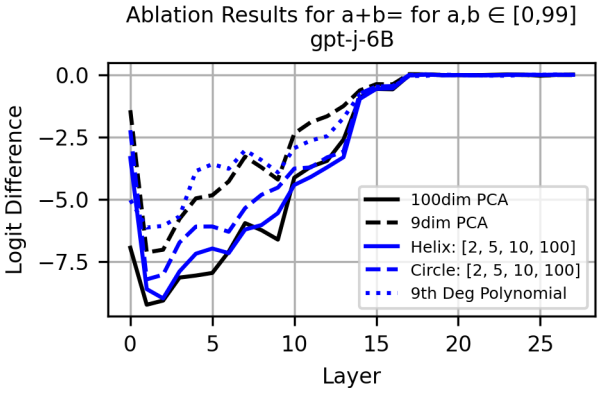


Figure 21. We find that ablating the helix dimensions from the residual stream is roughly as destructive as ablating the entire layer, providing additional evidence that GPT-J uses a numerical helix to do addition.

as "receiver" nodes, and patch in the path between all upstream components and the receiver node to determine what components each head relies on to achieve its causal effect. We find that $a + b$ heads rely predominantly on upstream MLPs, while mixed heads use both a, b heads and upstream MLPs. This indicates that mixed heads may have some role in creating $\text{helix}(a + b)$.

D.2. MLPs and Neurons

In Fig. 30, we see that patching $k = 11$ MLPs achieves 95% of the effect of patching all MLPs. We consider these MLPs to be circuit MLPs. Zooming in at the neuron level, we find that roughly 1% of neurons are required to achieve an 80% success rate on prompts while mean ablating all other neurons (Fig. 31). Note the use of accuracy over logit difference as a metric in this case. Fig. 31 shows that ablating some neurons actually *helps* performance as measured by logit difference, while hurting accuracy. To account for this seemingly contradictory result, we hypothesize that ablating some neurons asymmetrically boosts the answer token across prompts, such that some prompts are boosted significantly while other prompts are not affected. We do not investigate this further as it is not a major part of our argument and instead use an accuracy threshold.

When plotting the distribution of top neurons across layers in Fig. 32, we find that almost 75% of top neurons are located in the $k = 11$ circuit MLPs we have identified. We then path patch each of these neurons to calculate their direct effect. In Fig. 33, we see that roughly 700 neurons

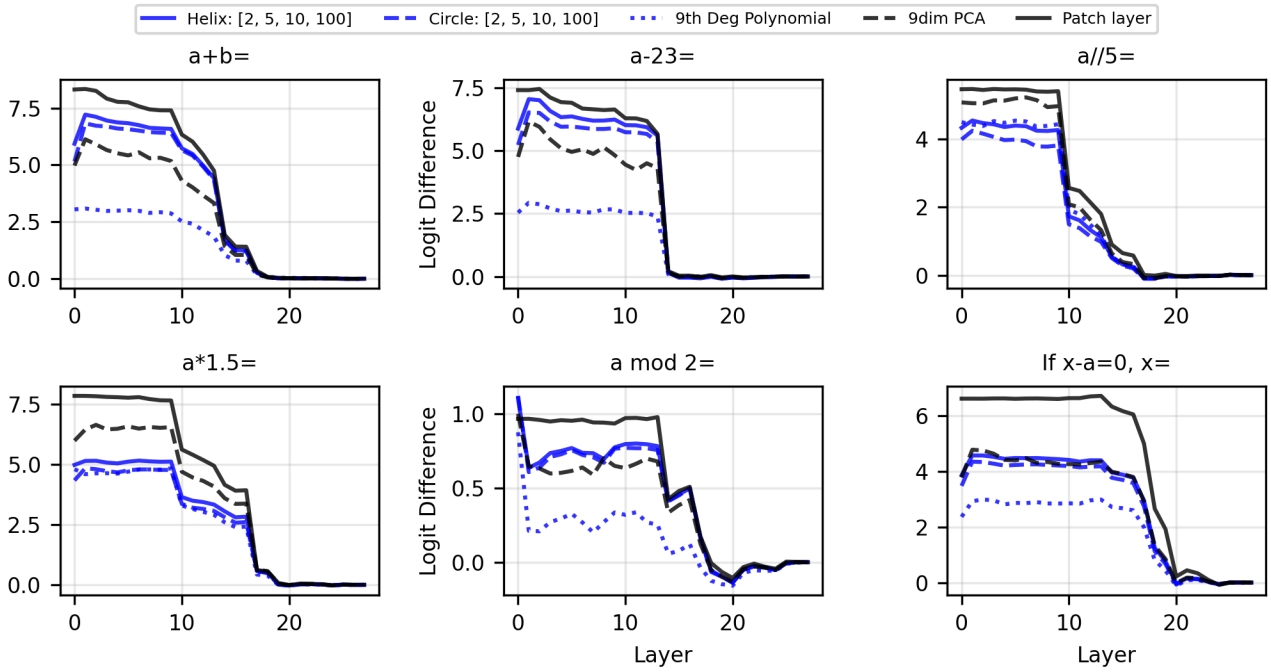


Figure 22. When testing our helical fit on other numerical tasks, we find that it performs competently, but does not always outperform the PCA baseline. This implies that additional structure may be present in numerical representations.

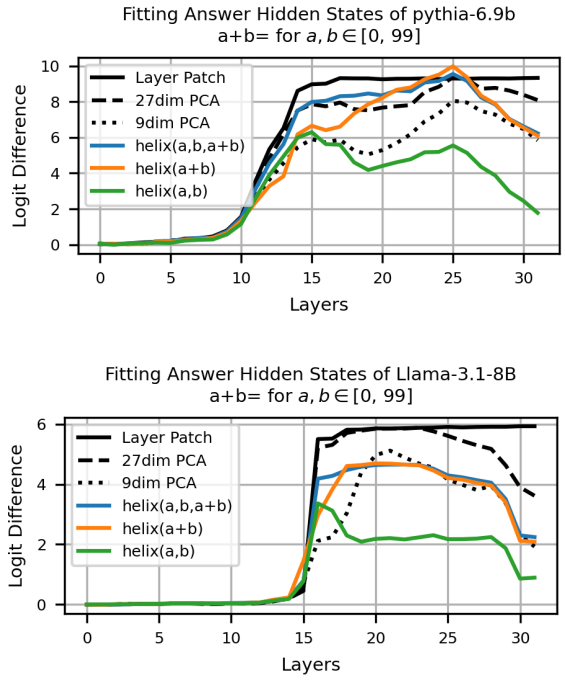


Figure 23. We present helical fits on the last token for Pythia-6.9B and Llama3.1-8B. The fits are weaker for Llama3.1-8B, potentially indicating the use of non-Clock algorithms.

are required to achieve 80% of the direct effect of patching in all $k = 4587$ top neurons. 84% of the top DE neurons occur after layer 18, which corresponds with our claim that MLPs 19-27 primarily write the correct answer to logits.

When we Fourier decompose the $k = 4587$ top neurons' preactivations with respect to the value of $a + b$ in Fig. 34, we see spikes at periods $T = [2, 5, 10, 100]$. These are the exact periods of our helix parameterization. To leverage this intuition, we fit the neuron preactivation patterns using the helix inspired functional form detailed in Eq. 2. We use a stochastic gradient descent optimizer with $\text{lr} = 1e{-}1, \text{epochs} = 2500$ and a cosine annealing learning rate scheduler to minimize the mean squared error of the fit. In Fig. 35, we see that more important neurons with larger total effect are fit better with this functional form, as measured by normalized root mean square error (NRMSE).

E. Why Use the Clock Algorithm at All?

We conjecture that LLMs use the Clock algorithm as a form of robust, error correcting code. If LLMs used a linear representation of numbers to do addition, that representation would have to be extremely precise to be effective.

To preliminarily test this conjecture, we take the first 50 PCA dimensions of the number representations for $a \in [0, 99]$ in GPT-J after layer 0 and fit a line ℓ to it. The resulting line

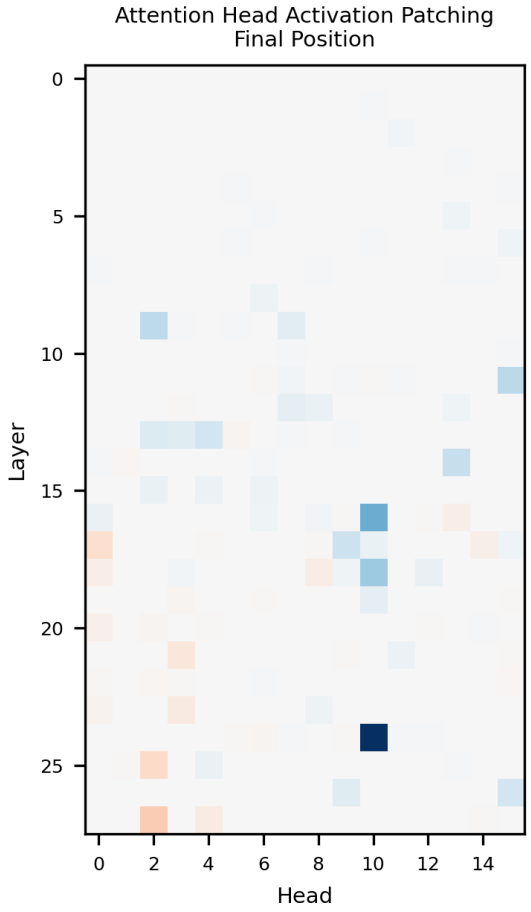


Figure 24. A sparse set of attention heads have causal effects on the output when patched (total effect visualized).

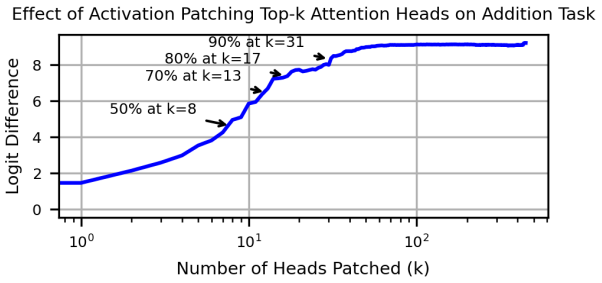


Figure 25. Patching $k = 20$ heads at once restores more than 80% of model behavior of patching all $k = 448$ heads.

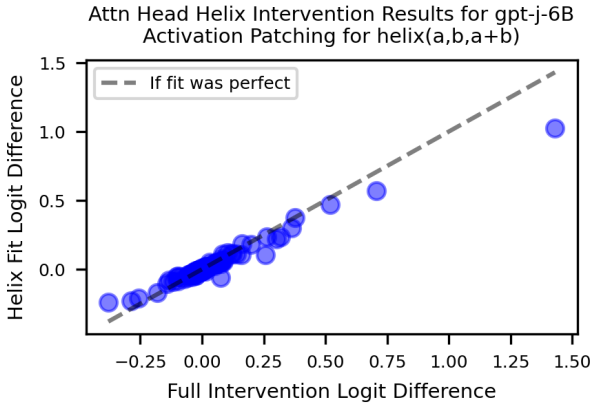


Figure 26. Attention head outputs are well modeled with $\text{helix}(a, b, a + b)$. Total effect shown.

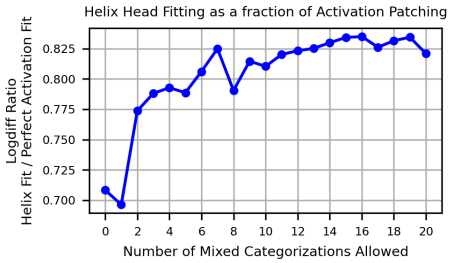


Figure 27. $m = 4$ mixed heads are sufficient to achieve 80% of effect of patching in all $k = 20$ heads normally. Even using $m = 0$ mixed heads achieves 70% of the effect.